
Directory.tla – formal specification AND safety proofs of the User- intent decision procedure for *MarkdownToLaTeX* 1.0.0.

Subject. The User runs *MarkdownToLaTeX* 1.0.0 from a working directory *DIR*. Before any conversion can begin, *MarkdownToLaTeX* decides whether *DIR* is “ready” at the file-system layer. Three flags carry the evidence:

1. *is_dir* $\$DIR$ exists and is a directory;
2. *has_preferences_dir* $\$DIR/preferences$ exists and is a directory;
3. *has_JSON_preferences* $\$DIR/preferences/preferences.json$ exists and is a regular file.

Validation of the *JSON* *content* (schema keys, legal values for “document class”, etc.) is OUT OF SCOPE for this iteration; it is left for a later kaizen step.

Two records, two semantics.

directory : Knowledge is the * trial verdict * .Each flag is
TRUE unless and until a check disproves it.
Init : all TRUE(no disproof yet).Transitions
flip TRUE $\rightarrow$ FALSE only and never back.
check : Knowledge is the * inquiry log * .Each flag is
TRUE iff the corresponding check has been run.
Init : all FALSE.Transitions flip FALSE $\rightarrow$ TRUE
only and never back.

The two records share the structural type *Knowledge* ; the
distinct meanings live in the actions, not the type.

Verdicts.OK $\triangleq$ AllChecked $\wedge$ NothingHasBeenDisproved
NOT_OK $\triangleq$ $\exists n \in 1..3 : hasBeenDisproved(n)$
Headline(T8) : the verdicts are mutually exclusive --
Spec $\Rightarrow \Box \neg (OK \wedge NOT_OK)$
so whenever the procedure halts, OK XOR NOT_OK holds.

Liveness.This module proves SAFETY only.No fairness clause is
added, no WF, no liveness theorem.Termination of the chain
is not proven here(and is not needed for the headline XOR
property since XOR reduces to disjointness under the
definitions of OK and NOT_OK).

Author.Jean – Gabriel Cordier, with Claude.

Origin.Earlier draft Tue Aug 08 2023 by gcordier.

This file.Kaizen step 1.3.7 – three – step file – system chain with
safety theorems T1 – T8 and complete TLAPS proof bodies.

EXTENDS *TLAPS*, *FiniteSets*, *Naturals*

VARIABLES

directory -- proof flags accumulated about *DIR*.Each leaf is TRUE
initially and may flip to FALSE only via a FAIL action.

Flags never flip back.

check *-- inquiry flags. Each leaf is FALSE initially and may flip to TRUE only when the corresponding check is performed. Flags never flip back.*

VARIABLES *directory, check*

vars $\triangleq \langle \textit{directory}, \textit{check} \rangle$

TYPE

A single shared structural type. Distinct meanings (verdict vs. inquiry log) are carried by the actions, not the type.

Knowledge $\triangleq [$
 is_dir : BOOLEAN ,
 has_preferences_dir : BOOLEAN ,
 has_JSON_preferences : BOOLEAN
 $]$

TypeOK $\triangleq$
 $\wedge \textit{directory} \in \textit{Knowledge}$
 $\wedge \textit{check} \in \textit{Knowledge}$

PROOF – FLAG ACCESSORS

Convention : indices 1 .. 3 line up with the three flags.

isTrue(n) $\triangleq$
 CASE $n = 1 \rightarrow \textit{directory.is_dir}$
 □ $n = 2 \rightarrow \textit{directory.has_preferences_dir}$
 □ $n = 3 \rightarrow \textit{directory.has_JSON_preferences}$
 □ OTHER $\rightarrow \text{FALSE}$

NothingHasBeenDisproved $\triangleq \forall n \in 1 .. 3 : \textit{isTrue}(n)$
hasBeenDisproved(n) $\triangleq \neg \textit{isTrue}(n)$
AllDisproved $\triangleq \forall n \in 1 .. 3 : \textit{hasBeenDisproved}(n)$

hasBeenChecked(n) $\triangleq$
 CASE $n = 1 \rightarrow \textit{check.is_dir}$
 □ $n = 2 \rightarrow \textit{check.has_preferences_dir}$

- $n = 3 \rightarrow \text{check.has_JSON_preferences}$
- OTHER $\rightarrow \text{FALSE}$

$\text{AllChecked} \triangleq \forall n \in 1 \dots 3 : \text{hasBeenChecked}(n)$
 $\text{hasNotBeenChecked}(n) \triangleq \neg \text{hasBeenChecked}(n)$
 $\text{NothingHasBeenChecked} \triangleq \forall n \in 1 \dots 3 : \text{hasNotBeenChecked}(n)$

VERDICT PREDICATES

$\text{OK} \triangleq \text{AllChecked} \wedge \text{NothingHasBeenDisproved}$
 $\text{NOT_OK} \triangleq \exists n \in 1 \dots 3 : \text{hasBeenDisproved}(n)$

INIT

$\text{InitDirectory} \triangleq$
 $\quad \wedge \text{directory.is_dir} = \text{TRUE}$
 $\quad \wedge \text{directory.has_preferences_dir} = \text{TRUE}$
 $\quad \wedge \text{directory.has_JSON_preferences} = \text{TRUE}$

$\text{InitCheck} \triangleq$
 $\quad \wedge \text{check.is_dir} = \text{FALSE}$
 $\quad \wedge \text{check.has_preferences_dir} = \text{FALSE}$
 $\quad \wedge \text{check.has_JSON_preferences} = \text{FALSE}$

$\text{Init} \triangleq$
 $\quad \wedge \text{directory} \in \text{Knowledge}$
 $\quad \wedge \text{check} \in \text{Knowledge}$
 $\quad \wedge \text{InitDirectory}$
 $\quad \wedge \text{InitCheck}$

SUCCESS/FAIL ACTIONS

Each step has two action shapes :
 $\text{Next} * (\text{success})$: leaves directory unchanged, sets the matching
 $\quad \text{check flag to TRUE.}$
 $\text{NextNo} * (\text{fail})$: flips the matching directory flag to FALSE AND
 $\quad \text{sets the matching check flag to TRUE.}$
Every action requires $\text{NothingHasBeenDisproved}$ as part of its guard, so
no further check fires after the first failure.

Step 1 : DIR is a directory.
 $\text{NecessaryConditionDir} \triangleq$

$$\begin{aligned}
& \wedge \text{NothingHasBeenChecked} \\
& \wedge \text{NothingHasBeenDisproved} \\
\text{NextHasDir} & \triangleq \\
& \wedge \text{NecessaryConditionDir} \\
& \wedge \text{UNCHANGED } \text{directory} \\
& \wedge \text{check}' = [\text{check} \text{ EXCEPT } !.\text{is_dir} = \text{TRUE}] \\
\text{NextHasNoDir} & \triangleq \\
& \wedge \text{NecessaryConditionDir} \\
& \wedge \text{directory}' = [\text{directory} \text{ EXCEPT } !.\text{is_dir} = \text{FALSE}] \\
& \wedge \text{check}' = [\text{check} \text{ EXCEPT } !.\text{is_dir} = \text{TRUE}] \\
\text{Step 2 : DIR/preferences is a directory.} \\
\text{NecessaryConditionPreferencesDir} & \triangleq \\
& \wedge \text{hasBeenChecked}(1) \\
& \wedge \text{hasNotBeenChecked}(2) \\
& \wedge \text{NothingHasBeenDisproved} \\
\text{NextHasPreferencesDir} & \triangleq \\
& \wedge \text{NecessaryConditionPreferencesDir} \\
& \wedge \text{UNCHANGED } \text{directory} \\
& \wedge \text{check}' = [\text{check} \text{ EXCEPT } !.\text{has_preferences_dir} = \text{TRUE}] \\
\text{NextHasNoPreferencesDir} & \triangleq \\
& \wedge \text{NecessaryConditionPreferencesDir} \\
& \wedge \text{directory}' = [\text{directory} \text{ EXCEPT } !.\text{has_preferences_dir} = \text{FALSE}] \\
& \wedge \text{check}' = [\text{check} \text{ EXCEPT } !.\text{has_preferences_dir} = \text{TRUE}] \\
\text{Step 3 : DIR/preferences/preferences.json is a regular file.} \\
\text{NecessaryJsonPreferences} & \triangleq \\
& \wedge \text{hasBeenChecked}(1) \\
& \wedge \text{hasBeenChecked}(2) \\
& \wedge \text{hasNotBeenChecked}(3) \\
& \wedge \text{NothingHasBeenDisproved} \\
\text{NextHasJsonPreferences} & \triangleq \\
& \wedge \text{NecessaryJsonPreferences} \\
& \wedge \text{UNCHANGED } \text{directory} \\
& \wedge \text{check}' = [\text{check} \text{ EXCEPT } !.\text{has_JSON_preferences} = \text{TRUE}] \\
\text{NextHasNoJsonPreferences} & \triangleq \\
& \wedge \text{NecessaryJsonPreferences} \\
& \wedge \text{directory}' = [\text{directory} \text{ EXCEPT } !.\text{has_JSON_preferences} = \text{FALSE}] \\
& \wedge \text{check}' = [\text{check} \text{ EXCEPT } !.\text{has_JSON_preferences} = \text{TRUE}]
\end{aligned}$$

NO – DEADLOCK STUTTER ACTIONS

Once the chain has halted (success at step 3, or any failure), no proper $Next^* / NextNo^*$ action is enabled. The four guards below describe the four halt configurations; the body of each is the same $no - op(UNCHANGED\ directory \wedge UNCHANGED\ check)$, so all four collapse to a single state – transition shape in any safety proof.

$$\begin{aligned} NextNoDeadLockCriterion &\triangleq \\ &\wedge UNCHANGED\ directory \\ &\wedge UNCHANGED\ check \end{aligned}$$

$$\begin{aligned} NoDeadlockNextOK &\triangleq \\ &\wedge NothingHasBeenDisproved \\ &\wedge AllChecked \end{aligned}$$

$$\begin{aligned} NoDeadlockNextHasNoDir &\triangleq \\ &\wedge hasBeenChecked(1) \\ &\wedge hasNotBeenChecked(2) \\ &\wedge hasBeenDisproved(1) \end{aligned}$$

$$\begin{aligned} NoDeadlockNextHasNoPreferencesDir &\triangleq \\ &\wedge hasBeenChecked(1) \\ &\wedge hasBeenChecked(2) \\ &\wedge hasNotBeenChecked(3) \\ &\wedge isTrue(1) \\ &\wedge hasBeenDisproved(2) \end{aligned}$$

$$\begin{aligned} NoDeadlockNextHasNoJsonPreferences &\triangleq \\ &\wedge AllChecked \\ &\wedge isTrue(1) \\ &\wedge isTrue(2) \\ &\wedge hasBeenDisproved(3) \end{aligned}$$

$$\begin{aligned} NoDeadlockNext &\triangleq \\ &\wedge NextNoDeadLockCriterion \\ &\wedge (\vee NoDeadlockNextOK \\ &\quad \vee NoDeadlockNextHasNoDir \\ &\quad \vee NoDeadlockNextHasNoPreferencesDir \\ &\quad \vee NoDeadlockNextHasNoJsonPreferences) \end{aligned}$$

NEXT

$$\begin{aligned} Next &\triangleq \\ &\vee NextHasDir \end{aligned}$$

$\vee \text{NextHasNoDir}$
 $\vee \text{NextHasPreferencesDir}$
 $\vee \text{NextHasNoPreferencesDir}$
 $\vee \text{NextHasJsonPreferences}$
 $\vee \text{NextHasNoJsonPreferences}$
 $\vee \text{NoDeadlockNext}$

SPEC

Safety only. No fairness clause : liveness is out of scope.

$$\text{Spec} \triangleq \text{Init} \wedge \Box[\text{Next}]_{\text{vars}}$$

CHAIN INVARIANTS

Five state predicates that together capture every safety property of interest. They are bundled into Inv and proven simultaneously by induction in theorem T_Inv.

$$\begin{aligned} \text{ChainOrderCheck} &\triangleq \\ &\wedge \text{hasBeenChecked}(2) \Rightarrow \text{hasBeenChecked}(1) \\ &\wedge \text{hasBeenChecked}(3) \Rightarrow \text{hasBeenChecked}(2) \end{aligned}$$

$$\begin{aligned} \text{Soundness} &\triangleq \\ &\wedge \text{hasBeenChecked}(2) \Rightarrow \text{isTrue}(1) \\ &\wedge \text{hasBeenChecked}(3) \Rightarrow \text{isTrue}(2) \end{aligned}$$

$$\begin{aligned} \text{DisprovalImpliesChecked} &\triangleq \\ &\forall n \in 1 \dots 3 : \text{hasBeenDisproved}(n) \Rightarrow \text{hasBeenChecked}(n) \end{aligned}$$

$$\text{AtLeastOneDisproved} \triangleq \exists n \in 1 \dots 3 : \text{hasBeenDisproved}(n)$$

$$\begin{aligned} \text{AtLeastTwoDisproved} &\triangleq \exists m \in 1 \dots 3 : (\\ &\quad \exists n \in 1 \dots 3 \setminus \{m\} : \\ &\quad \quad \wedge \text{hasBeenDisproved}(m) \\ &\quad \quad \wedge \text{hasBeenDisproved}(n) \\ &\quad) \end{aligned}$$

$$\text{AtMostOneDisproved} \triangleq \neg \text{AtLeastTwoDisproved}$$

$$\begin{aligned} &\wedge \neg(\text{hasBeenDisproved}(1) \wedge \text{hasBeenDisproved}(2)) \\ &\wedge \neg(\text{hasBeenDisproved}(2) \wedge \text{hasBeenDisproved}(3)) \\ &\wedge \neg(\text{hasBeenDisproved}(1) \wedge \text{hasBeenDisproved}(3)) \end{aligned}$$

$$\text{VerdictsExclusive} \triangleq \neg(\text{OK} \wedge \text{NOT_OK})$$

$$\text{Inv} \triangleq$$

$\wedge TypeOK$
 $\wedge ChainOrderCheck$
 $\wedge Soundness$
 $\wedge DisprovalImpliesChecked$
 $\wedge AtMostOneDisproved$

MONOTONICITY STEP – PROPERTIES

Step – level(relating current and primed state)properties used by the monotonicity theorems T6 and T7.

$DirectoryMonotoneStep \triangleq$
 $\forall n \in 1 \dots 3 : hasBeenDisproved(n) \Rightarrow hasBeenDisproved(n)'$

$CheckMonotoneStep \triangleq$
 $\forall n \in 1 \dots 3 : hasBeenChecked(n) \Rightarrow hasBeenChecked(n)'$

THEOREMS

$T_Inv \ Spec \Rightarrow \Box Inv$
 $T1 \ Spec \Rightarrow \Box TypeOK$
 $T2 \ Spec \Rightarrow \Box ChainOrderCheck$
 $T3 \ Spec \Rightarrow \Box Soundness$
 $T4 \ Spec \Rightarrow \Box DisprovalImpliesChecked$
 $T5 \ Spec \Rightarrow \Box AtMostOneDisproved$
 $T6 \ Spec \Rightarrow \Box [DirectoryMonotoneStep]_vars$
 $T7 \ Spec \Rightarrow \Box [CheckMonotoneStep]_vars$
 $T8 \ Spec \Rightarrow \Box VerdictsExclusive$

T_Inv : the master inductive invariant.

THEOREM $T_Inv \triangleq Spec \Rightarrow \Box Inv$
 $\langle 1 \rangle 1. Init \Rightarrow Inv$
 $\langle 2 \rangle 1. Init \Rightarrow TypeOK$
 BY DEF $Init, InitDirectory, InitCheck, TypeOK, Knowledge$
 $\langle 2 \rangle 2. Init \Rightarrow ChainOrderCheck$
 BY DEF $Init, InitCheck, ChainOrderCheck, hasBeenChecked$
 $\langle 2 \rangle 3. Init \Rightarrow Soundness$
 BY DEF $Init, InitCheck, Soundness, hasBeenChecked$
 $\langle 2 \rangle 4. Init \Rightarrow DisprovalImpliesChecked$

BY DEF *Init*, *InitDirectory*, *DisprovalImpliesChecked*,
 hasBeenDisproved, *isTrue*
 ⟨2⟩5. *Init* $\Rightarrow$ *AtMostOneDisproved*
 BY DEF *Init*, *InitDirectory*, *AtMostOneDisproved*, *AtLeastTwoDisproved*,
 hasBeenDisproved, *isTrue*
 ⟨2⟩6. QED
 BY ⟨2⟩1, ⟨2⟩2, ⟨2⟩3, ⟨2⟩4, ⟨2⟩5 DEF *Inv*
 ⟨1⟩2. *Inv* $\wedge$ [*Next*]_{vars} $\Rightarrow$ *Inv'*
 ⟨2⟩ SUFFICES ASSUME *Inv*, [*Next*]_{vars} PROVE *Inv'*
 OBVIOUS
 ⟨2⟩1. CASE *NextHasDir*
 ⟨3⟩a. *directory'* = *directory*
 BY ⟨2⟩1 DEF *NextHasDir*
 ⟨3⟩b. *check'* = [*check* EXCEPT !.is_dir = TRUE]
 BY ⟨2⟩1 DEF *NextHasDir*
 ⟨3⟩c. *NothingHasBeenChecked*
 BY ⟨2⟩1 DEF *NextHasDir*, *NecessaryConditionDir*
 ⟨3⟩d. *NothingHasBeenDisproved*
 BY ⟨2⟩1 DEF *NextHasDir*, *NecessaryConditionDir*
 ⟨3⟩e. *TypeOK'*
 BY ⟨3⟩a, ⟨3⟩b DEF *Inv*, *TypeOK*, *Knowledge*
 ⟨3⟩f. *ChainOrderCheck'*
 BY ⟨3⟩b, ⟨3⟩c DEF *ChainOrderCheck*, *hasBeenChecked*,
 NothingHasBeenChecked, *hasNotBeenChecked*
 ⟨3⟩g. *Soundness'*
 BY ⟨3⟩b, ⟨3⟩c DEF *Soundness*, *hasBeenChecked*,
 NothingHasBeenChecked, *hasNotBeenChecked*
 ⟨3⟩h. *DisprovalImpliesChecked'*
 BY ⟨3⟩a, ⟨3⟩d DEF *DisprovalImpliesChecked*, *hasBeenDisproved*,
 isTrue, *NothingHasBeenDisproved*
 ⟨3⟩i. *AtMostOneDisproved'*
 BY ⟨3⟩a, ⟨3⟩d DEF *AtMostOneDisproved*, *hasBeenDisproved*, *AtLeastTwoDisproved*,
 isTrue, *NothingHasBeenDisproved*
 ⟨3⟩j. QED
 BY ⟨3⟩e, ⟨3⟩f, ⟨3⟩g, ⟨3⟩h, ⟨3⟩i DEF *Inv*
 ⟨2⟩2. CASE *NextHasNoDir*
 ⟨3⟩a. *directory'* = [*directory* EXCEPT !.is_dir = FALSE]
 BY ⟨2⟩2 DEF *NextHasNoDir*
 ⟨3⟩b. *check'* = [*check* EXCEPT !.is_dir = TRUE]
 BY ⟨2⟩2 DEF *NextHasNoDir*
 ⟨3⟩c. *NothingHasBeenChecked*
 BY ⟨2⟩2 DEF *NextHasNoDir*, *NecessaryConditionDir*
 ⟨3⟩d. *NothingHasBeenDisproved*
 BY ⟨2⟩2 DEF *NextHasNoDir*, *NecessaryConditionDir*
 ⟨3⟩e. *TypeOK'*

(3)l. QED
 BY (3)g, (3)h, (3)i, (3)j, (3)k DEF *Inv*
 (2)4.CASE *NextHasNoPreferencesDir*
 (3)a. *directory'* = [*directory* EXCEPT !*.has_preferences_dir* = FALSE]
 BY (2)4 DEF *NextHasNoPreferencesDir*
 (3)b. *check'* = [*check* EXCEPT !*.has_preferences_dir* = TRUE]
 BY (2)4 DEF *NextHasNoPreferencesDir*
 (3)c. *hasBeenChecked*(1)
 BY (2)4 DEF *NextHasNoPreferencesDir*,
 NecessaryConditionPreferencesDir
 (3)d. *hasNotBeenChecked*(2)
 BY (2)4 DEF *NextHasNoPreferencesDir*,
 NecessaryConditionPreferencesDir
 (3)e. *NothingHasBeenDisproved*
 BY (2)4 DEF *NextHasNoPreferencesDir*,
 NecessaryConditionPreferencesDir
 (3)f. *hasNotBeenChecked*(3)
 BY (3)d DEF *Inv*, *ChainOrderCheck*, *hasBeenChecked*,
 hasNotBeenChecked
 (3)g. *TypeOK'*
 BY (3)a, (3)b DEF *Inv*, *TypeOK*, *Knowledge*
 (3)h. *ChainOrderCheck'*
 BY (3)b, (3)c, (3)f DEF *ChainOrderCheck*, *hasBeenChecked*,
 hasNotBeenChecked
 (3)i. *Soundness'*
 BY (3)a, (3)b, (3)e, (3)f DEF *Soundness*, *hasBeenChecked*,
 hasNotBeenChecked, *isTrue*,
 NothingHasBeenDisproved
 (3)j. *DisprovalImpliesChecked'*
 BY (3)a, (3)b, (3)e DEF *DisprovalImpliesChecked*,
 hasBeenDisproved, *hasBeenChecked*,
 isTrue, *NothingHasBeenDisproved*
 (3)k. *AtMostOneDisproved'*
 BY (3)a, (3)e DEF *AtMostOneDisproved*, *hasBeenDisproved*, *AtLeastTwoDisproved*,
 isTrue, *NothingHasBeenDisproved*
 (3)l. QED
 BY (3)g, (3)h, (3)i, (3)j, (3)k DEF *Inv*
 (2)5.CASE *NextHasJsonPreferences*
 (3)a. *directory'* = *directory*
 BY (2)5 DEF *NextHasJsonPreferences*
 (3)b. *check'* = [*check* EXCEPT !*.has_JSON_preferences* = TRUE]
 BY (2)5 DEF *NextHasJsonPreferences*
 (3)c. *hasBeenChecked*(1)
 BY (2)5 DEF *NextHasJsonPreferences*, *NecessaryJsonPreferences*
 (3)d. *hasBeenChecked*(2)

BY {2}5 DEF NextHasJsonPreferences, NecessaryJsonPreferences
{3}e. NothingHasBeenDisproved
BY {2}5 DEF NextHasJsonPreferences, NecessaryJsonPreferences
{3}f. TypeOK'
BY {3}a, {3}b DEF Inv, TypeOK, Knowledge
{3}g. ChainOrderCheck'
BY {3}b, {3}c, {3}d DEF ChainOrderCheck, hasBeenChecked
{3}h. Soundness'
BY {3}a, {3}b, {3}e DEF Soundness, hasBeenChecked, isTrue,
NothingHasBeenDisproved
{3}i. DisprovalImpliesChecked'
BY {3}a, {3}e DEF DisprovalImpliesChecked, hasBeenDisproved,
isTrue, NothingHasBeenDisproved
{3}j. AtMostOneDisproved'
BY {3}a, {3}e DEF AtMostOneDisproved, AtLeastTwoDisproved, hasBeenDisproved,
isTrue, NothingHasBeenDisproved
{3}k. QED
BY {3}f, {3}g, {3}h, {3}i, {3}j DEF Inv

{2}6.CASE NextHasNoJsonPreferences
{3}a. directory' = [directory EXCEPT !.has_JSON_preferences = FALSE]
BY {2}6 DEF NextHasNoJsonPreferences
{3}b. check' = [check EXCEPT !.has_JSON_preferences = TRUE]
BY {2}6 DEF NextHasNoJsonPreferences
{3}c. hasBeenChecked(1)
BY {2}6 DEF NextHasNoJsonPreferences, NecessaryJsonPreferences
{3}d. hasBeenChecked(2)
BY {2}6 DEF NextHasNoJsonPreferences, NecessaryJsonPreferences
{3}e. NothingHasBeenDisproved
BY {2}6 DEF NextHasNoJsonPreferences, NecessaryJsonPreferences
{3}f. TypeOK'
BY {3}a, {3}b DEF Inv, TypeOK, Knowledge
{3}g. ChainOrderCheck'
BY {3}b, {3}c, {3}d DEF ChainOrderCheck, hasBeenChecked
{3}h. Soundness'
BY {3}a, {3}b, {3}e DEF Soundness, hasBeenChecked, isTrue,
NothingHasBeenDisproved
{3}i. DisprovalImpliesChecked'
BY {3}a, {3}b, {3}e DEF DisprovalImpliesChecked,
hasBeenDisproved, hasBeenChecked,
isTrue, NothingHasBeenDisproved
{3}j. AtMostOneDisproved'
BY {3}a, {3}e DEF AtMostOneDisproved, AtLeastTwoDisproved, hasBeenDisproved, AtLeastTwoDisproved,
isTrue, NothingHasBeenDisproved
{3}k. QED
BY {3}f, {3}g, {3}h, {3}i, {3}j DEF Inv

$\langle 2 \rangle 7$.CASE *NoDeadlockNext*
 BY $\langle 2 \rangle 7$ DEF *NoDeadlockNext*, *NextNoDeadLockCriterion*, *Inv*,
TypeOK, *ChainOrderCheck*, *Soundness*,
DisprovalImpliesChecked, *AtMostOneDisproved*, *AtLeastTwoDisproved*,
hasBeenChecked, *hasBeenDisproved*, *isTrue*,
NothingHasBeenDisproved
 $\langle 2 \rangle 8$.CASE UNCHANGED *vars*
 BY $\langle 2 \rangle 8$ DEF *Inv*, *TypeOK*, *ChainOrderCheck*, *Soundness*,
DisprovalImpliesChecked, *AtMostOneDisproved*, *AtLeastTwoDisproved*,
hasBeenChecked, *hasBeenDisproved*, *isTrue*,
NothingHasBeenDisproved, *vars*
 $\langle 2 \rangle 9$. QED
 BY $\langle 2 \rangle 1$, $\langle 2 \rangle 2$, $\langle 2 \rangle 3$, $\langle 2 \rangle 4$, $\langle 2 \rangle 5$, $\langle 2 \rangle 6$, $\langle 2 \rangle 7$, $\langle 2 \rangle 8$ DEF *Next*
 $\langle 1 \rangle 3$. QED
 BY $\langle 1 \rangle 1$, $\langle 1 \rangle 2$, *PTL* DEF *Spec*

T1 – T5 : direct corollaries of T_Inv.

THEOREM $T1 \triangleq Spec \Rightarrow \Box TypeOK$
 BY *T_Inv*, *PTL* DEF *Inv*

THEOREM $T2 \triangleq Spec \Rightarrow \Box ChainOrderCheck$
 BY *T_Inv*, *PTL* DEF *Inv*

THEOREM $T3 \triangleq Spec \Rightarrow \Box Soundness$
 BY *T_Inv*, *PTL* DEF *Inv*

THEOREM $T4 \triangleq Spec \Rightarrow \Box DisprovalImpliesChecked$
 BY *T_Inv*, *PTL* DEF *Inv*

THEOREM $T5 \triangleq Spec \Rightarrow \Box AtMostOneDisproved$
 BY *T_Inv*, *PTL* DEF *Inv*

T6 : directory is monotone non – increasing.

THEOREM $T6 \triangleq Spec \Rightarrow \Box [DirectoryMonotoneStep]_{vars}$
 $\langle 1 \rangle 1$. $[Next]_{vars} \Rightarrow [DirectoryMonotoneStep]_{vars}$
 $\langle 2 \rangle$ SUFFICES ASSUME $[Next]_{vars}$ PROVE $[DirectoryMonotoneStep]_{vars}$
 OBVIOUS
 $\langle 2 \rangle 1$.CASE *NextHasDir*
 BY $\langle 2 \rangle 1$ DEF *NextHasDir*, *DirectoryMonotoneStep*, *hasBeenDisproved*,
isTrue
 $\langle 2 \rangle 2$.CASE *NextHasNoDir*
 BY $\langle 2 \rangle 2$ DEF *NextHasNoDir*, *NecessaryConditionDir*,
DirectoryMonotoneStep, *hasBeenDisproved*, *isTrue*,
NothingHasBeenDisproved
 $\langle 2 \rangle 3$.CASE *NextHasPreferencesDir*

```

    BY ⟨2⟩3 DEF NextHasPreferencesDir, DirectoryMonotoneStep,
              hasBeenDisproved, isTrue
  ⟨2⟩4.CASE NextHasNoPreferencesDir
    BY ⟨2⟩4 DEF NextHasNoPreferencesDir,
              NecessaryConditionPreferencesDir,
              DirectoryMonotoneStep, hasBeenDisproved, isTrue,
              NothingHasBeenDisproved
  ⟨2⟩5.CASE NextHasJsonPreferences
    BY ⟨2⟩5 DEF NextHasJsonPreferences, DirectoryMonotoneStep,
              hasBeenDisproved, isTrue
  ⟨2⟩6.CASE NextHasNoJsonPreferences
    BY ⟨2⟩6 DEF NextHasNoJsonPreferences, NecessaryJsonPreferences,
              DirectoryMonotoneStep, hasBeenDisproved, isTrue,
              NothingHasBeenDisproved
  ⟨2⟩7.CASE NoDeadlockNext
    BY ⟨2⟩7 DEF NoDeadlockNext, NextNoDeadLockCriterion,
              DirectoryMonotoneStep, hasBeenDisproved, isTrue
  ⟨2⟩8.CASE UNCHANGED vars
    BY ⟨2⟩8 DEF DirectoryMonotoneStep, hasBeenDisproved, isTrue, vars
  ⟨2⟩9. QED
    BY ⟨2⟩1, ⟨2⟩2, ⟨2⟩3, ⟨2⟩4, ⟨2⟩5, ⟨2⟩6, ⟨2⟩7, ⟨2⟩8 DEF Next
  ⟨1⟩2. QED
    BY ⟨1⟩1, PTL DEF Spec

```

T7 : check is monotone non – decreasing.

THEOREM $T7 \triangleq Spec \Rightarrow \Box[CheckMonotoneStep]_{vars}$

```

  ⟨1⟩1.  $[Next]_{vars} \Rightarrow [CheckMonotoneStep]_{vars}$ 
  ⟨2⟩ SUFFICES ASSUME  $[Next]_{vars}$  PROVE  $[CheckMonotoneStep]_{vars}$ 
    OBVIOUS
  ⟨2⟩1.CASE NextHasDir
    BY ⟨2⟩1 DEF NextHasDir, CheckMonotoneStep, hasBeenChecked
  ⟨2⟩2.CASE NextHasNoDir
    BY ⟨2⟩2 DEF NextHasNoDir, CheckMonotoneStep, hasBeenChecked
  ⟨2⟩3.CASE NextHasPreferencesDir
    BY ⟨2⟩3 DEF NextHasPreferencesDir, CheckMonotoneStep,
              hasBeenChecked
  ⟨2⟩4.CASE NextHasNoPreferencesDir
    BY ⟨2⟩4 DEF NextHasNoPreferencesDir, CheckMonotoneStep,
              hasBeenChecked
  ⟨2⟩5.CASE NextHasJsonPreferences
    BY ⟨2⟩5 DEF NextHasJsonPreferences, CheckMonotoneStep,
              hasBeenChecked
  ⟨2⟩6.CASE NextHasNoJsonPreferences
    BY ⟨2⟩6 DEF NextHasNoJsonPreferences, CheckMonotoneStep,

```

hasBeenChecked

⟨2⟩7.CASE *NoDeadlockNext*
 BY ⟨2⟩7 DEF *NoDeadlockNext*, *NextNoDeadLockCriterion*,
 CheckMonotoneStep, *hasBeenChecked*

⟨2⟩8.CASE UNCHANGED *vars*
 BY ⟨2⟩8 DEF *CheckMonotoneStep*, *hasBeenChecked*, *vars*

⟨2⟩9. QED

BY ⟨2⟩1, ⟨2⟩2, ⟨2⟩3, ⟨2⟩4, ⟨2⟩5, ⟨2⟩6, ⟨2⟩7, ⟨2⟩8 DEF *Next*

⟨1⟩2. QED

BY ⟨1⟩1, *PTL* DEF *Spec*

T8 : the headline -- OK and NOT_OK never coexist.
Proof is purely propositional : NOT_OK ≡ ¬NothingHasBeenDisproved,
while OK requires NothingHasBeenDisproved.

THEOREM $T8 \triangleq Spec \Rightarrow \Box VerdictsExclusive$

⟨1⟩1. *Init* $\Rightarrow VerdictsExclusive$
 BY DEF *VerdictsExclusive*, *OK*, *NOT_OK*,
 NothingHasBeenDisproved, *hasBeenDisproved*

⟨1⟩2. $VerdictsExclusive \wedge [Next]_{vars} \Rightarrow VerdictsExclusive'$
 BY DEF *VerdictsExclusive*, *OK*, *NOT_OK*,
 NothingHasBeenDisproved, *hasBeenDisproved*

⟨1⟩3. QED

BY ⟨1⟩1, ⟨1⟩2, *PTL* DEF *Spec*

\ * *Modification History*
 \ * *Last modified Sat May 02 16 : 17 : 10 CEST 2026 by gcordier*
 \ * *Last modified Sat May 02 2026 by gcordier(kaizen 1.3.7)*
 \ * *Last modified Sat May 02 11 : 19 : 29 CEST 2026 by gcordier(kaizen 1.3.6)*